

Module 01 - Report

Stefano Vitale

1 Research for the Optimal Hash Function

The partition mapping kernel assigns each 64-bit key to a partition index in $[0, P)$. The choice of hash function impacts directly two things:

1. **distribution quality**: how evenly the keys are distributed across all partitions.
2. **computational speed**: the execution throughput of the mapping phase.

In the first phase the focus was selecting an optimal hash function capable of balancing the strict trade-off between **quality** and **speed**.

1.1 Distribution Quality

Five different kernels were selected for evaluation. To quantify their distribution quality over N keys and P partitions, two main metrics were analyzed:

- **Coefficient of Variation** ($CV = \sigma/\mu$): measures global spread across all partitions.
- **Overload Factor** ($OF = M/\mu$): ratio of the most loaded partition to the ideal average. Directly quantifies the worst-case that limits parallel execution (data skew).

To evaluate the robustness of each hash function against poor input entropy, keys were generated using four different distribution patterns: **Uniform** (random 64-bit keys), **Sequential** (auto-incrementing IDs), **Multiples** (keys that are exact multiples of 64), and **Duplicates** (low-entropy input with heavy repetition).

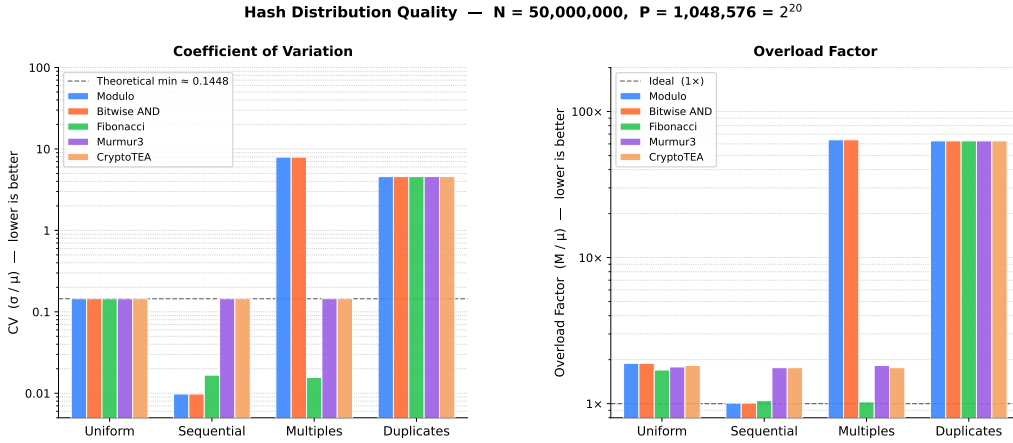


Figure 1: Coefficient of Variation and Overload Factor across 5 hash functions and 4 key patterns

As shown in Figure 1, *Modulo* and *Bitwise AND* collapse on the *Multiples* pattern, making them unsafe for workloads with non-random keys. Conversely, *Fibonacci*, *Murmur3* and *CryptoTEA* all handle all cases correctly. The *Duplicates* result remains consistent across all functions, confirming that identical inputs represent a physical limit of the dataset rather than a hashing failure.

1.2 Performance Comparison

After validating the distribution quality, the focus shifted to performance, evaluated on the **node09** of the SPM cluster:

1.3 Final Choice

Modulo and *Bitwise* were previously discarded due to bad distribution quality, and after performance comparison, also *CryptoTEA* was discarded, confirming that cryptographic functions do not fit the problem of this project (the cost of stronger mixing is lower throughput).

While *Fibonacci* exhibited the highest scalar throughput among the functions, it was discarded in favor of *Murmur3 finalizer (fmix32)*. The decisive factor was microarchitectural: *Fibonacci* relies on a 64-bit integer multiplication, which lacks a native packed instruction in the target AVX2. Instead, the official 32-bit *fmix32* maps easily onto native AVX2 32-bit instructions. As shown in Figure 3, avoiding 64-bit arithmetic allows the 32-bit variant of Murmur3 to achieve significantly higher throughput without compromising any distribution quality.

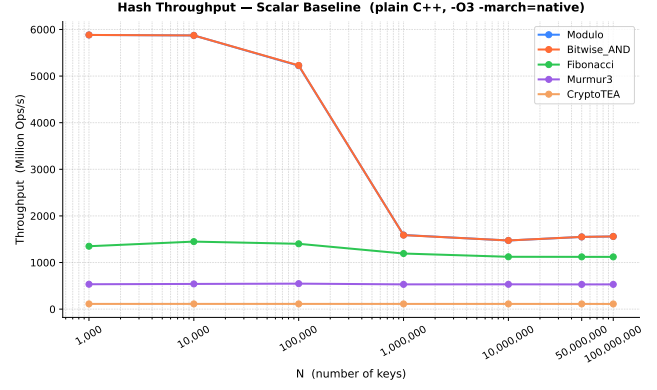


Figure 2: Scalar throughput (MOps/s). The performance drop illustrates the **Memory Wall**.

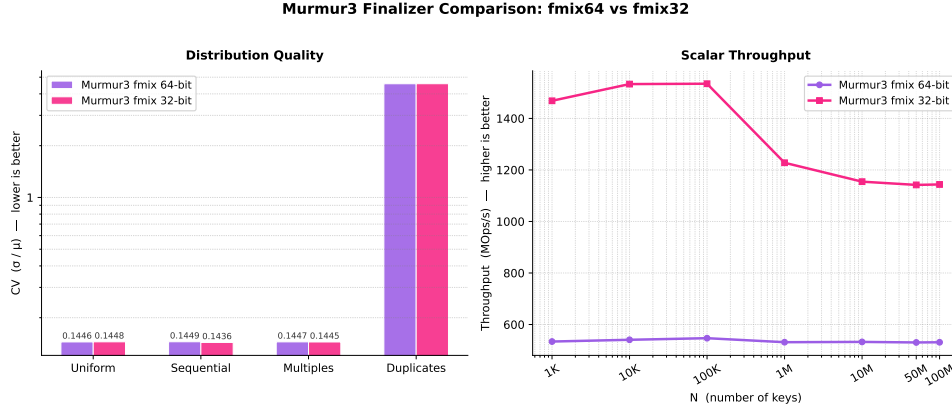


Figure 3: Throughput comparison between Murmur3 Finalizer 64-bit and 32-bit variants.

2 CPU-based Partition Mapping (Task 1 & 2)

In this part, all CPU-based implementations of the Murmur3 finalizer (fmix32) are compared across different aspects (sweeping N , sweeping P and Throughput).

2.1 Scalar Baseline and Auto-Vectorization

To balance execution speed and distribution quality, the implementation extracts and adapts only the 32-bit finalizer avalanche step (fmix32) of the full Murmur3 algorithm. It takes a 64-bit key and produces a 32-bit partition ID through an initial XOR-folding step, followed by the standard Murmur3 bitwise mixing sequence.

Since P is always a power of two, the final step is a fast right shift ($h \gg (32 - p_exp)$) rather than an expensive modulo operation. Two binaries are produced from the same source: **baseline** (compiled with `-O3 -fno-tree-vectorize`) and **autovec** (compiled with `-O3`).

2.2 Manual SIMD Vectorization

The first naive AVX2 implementation processed 4 keys at a time. However, since the initial step reduces each 64-bit key to 32 bits via XOR-folding, this approach would leave 50% of the SIMD lanes filled with garbage

data.

To prevent this, the algorithm processes a batch of 16 keys per iteration and utilizes fast in-lane shuffle (`_mm256_shuffle_epi32`) and unpack operations to densely pack the folded hashes into two fully saturated 256-bit registers. Consequently, the heavy arithmetic pipeline is executed with 100% lane utilization. The implementation is well detailed in the code (file `main_avx2.cpp`) with comments.

2.3 CPU Performance Evaluation

Keys (N)	Time (ms)			Speedup	
	Base	Auto	AVX2	Auto	AVX2
100k	0.21	0.06	0.08	3.26x	2.75x
1M	2.19	0.81	0.85	2.70x	2.55x
10M	21.92	8.85	9.07	2.47x	2.41x
50M	108.16	43.65	45.66	2.47x	2.36x
100M	217.19	86.89	90.37	2.49x	2.40x
150M	337.81	130.27	135.85	2.59x	2.48x
200M	450.21	171.50	183.11	2.62x	2.45x

Table 1: Scaling with N (fixed $P = 2^{20}$).

Parts (P)	Time (ms)		
	Base	Auto	AVX2
2^8	108.21	43.40	45.45
2^{12}	109.57	43.09	45.97
2^{16}	108.55	43.22	45.74
2^{20}	109.56	42.92	46.28
2^{24}	109.21	43.201	45.11

Table 2: Scaling with P ($N = 50M$).

The compiler auto-vectorization slightly outperforms the manual 256-bit AVX2 implementation on this platform. As analyzed in Section 5, this anomaly is strictly tied to the AMD Zen 1 microarchitecture, which penalizes 256-bit instructions. A 128-bit unrolled implementation fully closes this gap. Furthermore, tests sweeping the partition count P from 2^8 to 2^{24} (at fixed $N = 50M$) showed very low variance in execution time, confirming that the kernel throughput is perfectly independent of the output domain size.

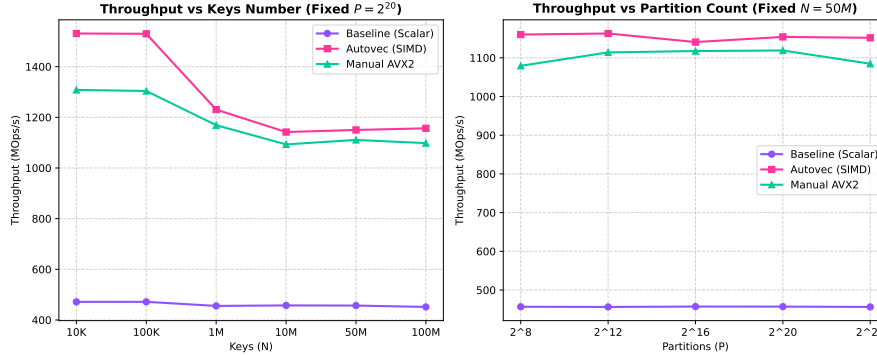


Figure 4: Throughput comparison and scaling behavior.

3 GPU-Based Partition Mapping (Task 3)

3.1 CUDA Kernel Architecture

The GPU porting adopts a straightforward, massively data-parallel strategy: mapping exactly one thread to each 64-bit key. To maximize modularity and maintain strict parity with the CPU versions, the core hashing logic was isolated into a `__device__` function. Following standard Device memory allocation and Host-to-Device data transfer, a `__global__` kernel orchestrates the execution. The kernel is launched across a 1D grid using a block size of 256 threads, a value chosen to ensure optimal warp scheduling and high occupancy on the Streaming Multiprocessors. Since the threads do not need to share intermediate results, the kernel relies entirely on fast private registers and Global Memory, with **no need for explicit Shared Memory synchronization**.

Keys (N)	Kernel Time (ms)	Total Transfer Time (ms)	Kernel Throughput (MOps/s)
100M	1.508	131.042	66.305,47
250M	3.752	327.262	66.636,51
500M	7.370	653.287	67.838,13
1000M	14.742	1518.722	67.833,01

Table 3: Kernel execution time and Host/Device transfer time for the CUDA implementation.

4 Correctness Verification

Correctness is verified in two ways. For all input patterns, all binaries dump the first 10,000 partitions IDs to file and a `diff` confirms element-by-element success. On the other hand, a global checksum over the full output array is printed and compared across the run. The details of compiling, execution and evaluation are better discussed in the `README.md` file. GCC auto-vectorization was confirmed via `-fopt-info-vec`, which reported: `loop vectorized using 16 byte vectors (main.cpp:60:26)`.

To guarantee repeatable and stable benchmarking, all performance metrics presented in this report are calculated as the **median of 7 independent runs**. For $N < 50M$, standard deviations remained below 0.1 ms. For larger inputs, occasional OS-level interference produced spikes up to 5 ms, which the median methodology absorbs without affecting the reported central tendency.

Keys (N)	Baseline stddev	Autovec stddev	AVX2 stddev	Cuda stddev
10M	0.048 ms	0.145 ms	0.171 ms	0.001 ms
50M	0.032 ms	0.018 ms	0.067 ms	0.001 ms
100M	3.240 ms	0.070 ms	0.019 ms	0.001 ms
150M	4.414 ms	4.278 ms	4.222 ms	0.002 ms
200M	3.217 ms	4.889 ms	5.295 ms	0.011 ms

Table 4: Standard deviation across 7 runs (fixed $P = 2^{20}$).

5 Conclusion

Architectural Justification for the Full Hash Join Pipeline. Simple hash functions (e.g., modulo or bitwise AND) are slightly faster, but they distribute non-uniform data poorly, causing data skew. In a full Partitioned Hash Join, this skew creates severe workload imbalances among threads. The Murmur3 finalizer (fmix32) was chosen as the best trade-off: it guarantees perfectly balanced partitions for any input, and its 32-bit operations map natively to AVX2 registers. This ensures fast vectorization now and a perfectly balanced workload for the future join phase.

Hardware-Aware Optimization: The 128-bit Unrolled Experiment While the required AVX2 (256-bit) implementation successfully accelerated the hashing phase, an in-depth architectural analysis of the target node (AMD EPYC 7001 series, Zen 1 microarchitecture) revealed a structural bottleneck. The Zen 1 architecture physically splits every 256-bit AVX2 instruction into two 128-bit micro-operations at runtime, incurring a structural decoding penalty.

To empirically test this hardware limitation and attempt to match the compiler’s -O3 auto-vectorization performance, an additional manual implementation was developed. This version uses only 128-bit registers (`__m128i`) to align with the physical execution width of the CPU. Furthermore, a 4-way manual loop unrolling strategy was applied. Processing 16 keys per iteration via 8 interleaved 128-bit registers effectively hides the instruction latency and maximizes Instruction-Level Parallelism (documented in the `main_avx.cpp` file).

Keys (N)	Execution Time (ms)			Speedup vs Base		
	Auto	AVX2	AVX	Auto	AVX2	AVX
50M	43.30	45.55	43.49	2.54x	2.41x	2.53x
100M	85.31	90.49	87.52	2.57x	2.42x	2.50x
250M	219.65	228.38	218.40	2.57x	2.47x	2.58x

Table 5: AVX vs AVX2 on Zen 1.